

```

1  #=====一些概括性的总结=====
2  #一、这里有哪些地方使用了std? 为什么?
3  #    (1) 用 std 来构造参数 (初始化)
4
5  #    (2) 用 std (方差) 来归一化激活值
6
7  #    (3) 用 std 来诊断--评估参数/梯度表现
8
9
10
11 #二、有了batch_norm层, 是否还需要对线性层参数进行归一化?
12 #    (1) 每一次经过不归一化的线性层, 输出的激活值都会更离散。那么多层叠加就会导致激活值过饱和
13 #        1. 如果不归一化W层, 那么假设Hin矩阵和W矩阵, 初始统计特征是: 均值为0, 方差为1。大小都为n*n
14 #        2. 假设Hout = Hin@W, 那么Hout的统计特征是怎么样的? ----均值不变, std变为n**0.5----统计学知识
15
16 #    (2) 我们设计的神经网络的结构如下: 线性层 ---> BN层 ---> tanh层 ---> 下一个线性层
17 #    (3) 分析没有归一化W参数情况下, forward过程, 激活值健康状况如何:
18
19
20 #    (4) 分析没有归一化W参数情况下, backward过程, 激活值健康状况如何:
21
22
23
24
25 #=====cover一些part1的细节还有内容总结=====
26 # Let's train a deeper network
27 # The classes we create here are the same API as nn.Module in PyTorch
28 import sys
29 import torch
30 import torch.nn.functional as F
31 import matplotlib.pyplot as plt
32 import random
33 from ipywidgets import interact, interactive, fixed, interact_manual
34 import ipywidgets as widgets
35 import scipy.stats as stats
36 import numpy as np
37
38 #=====init部分=====
39 #读入数据集, 并且对数据集进行分类
40 #构造字典, 定义上下文长度
41 words = open('Neural Network_Zero to hero/makemore/names.txt', 'r').read().splitlines()
42 chars = sorted(list(set(''.join(words))))
43 stoi = {s:i+1 for i,s in enumerate(chars)}
44 stoi['.'] = 0
45 itos = {i:s for s,i in stoi.items()}

```



```

91  def __call__(self, x):
92  #   __call__是一个魔术方法, 允许对象实例被像方法一样调用
93  #   传入的self是线性层本身, x是上一层的激活值, 用做这一层的参数
94  #   这个方法就是linear层的forward pass方法
95      self.out = x @ self.weight
96      if self.bias is not None:
97          self.out += self.bias
98      return self.out
99
100 #   提供外界接口, 方便地获取本linear层的参数
101 def parameters(self):
102     return [self.weight] + ([] if self.bias is None else [self.bias])
103
104
105
106 #=====此处是BatchNorm层的定义=====
107 class BatchNorm1d:
108
109 #   初始化方法, 生成BN层
110 def __init__(self, dim, eps=1e-5, momentum=0.1):
111     self.eps = eps
112     self.momentum = momentum
113     self.training = True
114
115     # parameters (trained with backprop), 也就是上一个part的Batch_gain
116     self.gamma = torch.ones(dim)
117     self.beta = torch.zeros(dim)
118
119     # buffers (trained with a running 'momentum update'),也就是上一个part的Batch_bias
120     self.running_mean = torch.zeros(dim)
121     self.running_var = torch.ones(dim)
122
123 #接受上一层输入的激励值, 由BN层进行归一化
124 def __call__(self, x):
125     # self是BN层自己, x为上一层传入的激励值
126     if self.training:
127         xmean = x.mean(0, keepdim=True) # batch mean
128         xvar = x.var(0, keepdim=True) # batch variance
129
130     else:
131         xmean = self.running_mean
132         xvar = self.running_var
133     xhat = (x - xmean) / torch.sqrt(xvar + self.eps) # normalize to unit variance
134     self.out = self.gamma * xhat + self.beta
135

```

```

136     #在训练的过程中不断更新逼近全局mean
137     if self.training:
138         with torch.no_grad():
139             self.running_mean = (1 - self.momentum) * self.running_mean + self.momentum * xmean
140             self.running_var = (1 - self.momentum) * self.running_var + self.momentum * xvar
141     return self.out
142
143     def parameters(self):
144         return [self.gamma, self.beta]
145
146
147     #=====激活函数层, 采用tanh=====
148     class Tanh:
149         def __call__(self, x):
150             self.out = torch.tanh(x)
151             return self.out
152         def parameters(self):
153             return []
154
155
156
157     #=====开始训练=====
158     n_embd = 30 # the dimensionality of the character embedding vectors
159     n_hidden = 1000 # the number of neurons in the hidden layer of the MLP
160     g = torch.Generator().manual_seed(2147483647) # for reproducibility
161
162     # (1) 初始化layers, 和特殊的嵌入矩阵C
163     C = torch.randn((vocab_size, n_embd), generator=g)
164     layers = [
165         Linear(n_embd * block_size, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
166         Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
167         Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
168         Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
169         Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
170         Linear(n_hidden, vocab_size, bias=False), BatchNorm1d(vocab_size),
171     ]
172
173
174     #这段代码作用: 训练开始前的"最后一层信心压制", 目的是让网络一开始输出接近均匀分布, 而不是"自信地瞎猜"。
175     with torch.no_grad():
176         # (2) 为什么最后一层的BN层的gamma值, 要乘一个0.1?
177         #输入index是-1, 表示选取list的最后一个对象
178         #这一层是紧邻着logits的最后一层, 把它整体乘 0.1, 等于把所有 27 个 logit 的幅度压缩 10 倍。
179         #原本的logits的值, 应该近似为标准正态分布, 那么交叉熵的期望大概是3.80-----看数学原理
180         #gamma乘了0.1. 相当于把激励值的输出的方差变成了原先的0.01倍. 于是交叉熵就变成了大约3.29-----具体计算看原理

```



```

226     for layer in layers:
227         layer.out.retain_grad() # AFTER_DEBUG: would take out retain_graph
228 # (5) 这部分是为了保存非叶子节点tensor内储存的grad, 为了调试使用
229 #     1.我们设定每一层的参数, 并不是由前面的层计算而来的, 本身就是总的神经网络tree的叶子
230 #     2.运算产生的中间结果, 比如 hpreact = x @ W + b.
231 #     反向传播时, 梯度只是流过它们, 算完传给下一层就释放了, 不会存进 .grad。
232 #     所以 hpreact.grad 是 None。
233 #     3.这里就是告诉Pytorch, 去储存每一层layer的输入的激励值的grad, 以及输出的激励值的grad
234     for p in parameters:
235 #         在更新梯度之前, 清空之前的存在的梯度, 再在当次的backward重新计算
236 #         因为parameter都是叶子tensor, 所以默认保存grad。所以每次更新之前需要我们来清空
237         p.grad = None
238     loss.backward()
239
240     #更新grad-----训练到后面的时候减小学习率
241     lr = 0.1 if i < 150000 else 0.01 # step learning rate decay
242     for p in parameters:
243         p.data += -lr * p.grad
244
245     #状态追踪
246     if i % 10000 == 0: # print every once in a while
247         print(f'{i:7d}/{max_steps:7d}: {loss.item():.4f}')
248     #记录每次iteration的对应loss
249     lossi.append(loss.log10().item())
250     with torch.no_grad():
251 # (6) 这部分是为了记录:
252 #     1.p in parameters里面的p, 指的是参数张量们。比如Parameters[0]就是矩阵C, 而不是矩阵C内的首个元素
253 #     2.所以这里实际上计算的是, 每一次iteration中, 某一个参数矩阵的“变化率”
254 #     3.并且按照我们当前的设计, 整个神经网络里面有19个参数tensor。
255 #     所以每一次iteration, 都会给ud列表加上一个列表I作为元素。
256 #     列表I内储存的元素是19个参数tensor中每一个参数tensor的(lr*p.grad).std() / p.data.std()).log10()值
257 #     4.所以ud的维度是i*19。i表示iteration的次数, 也就是训练次数
258     ud.append([((lr*p.grad).std() / p.data.std()).log10().item() for p in parameters])
259     if i % 100 == 1:
260         print(loss)
261
262
263 #=====用训练好的模型采样: 生成名字=====
264 def use(number_of_names=10):
265
266     # (1) 先把所有的BatchNorm层切成eval模式:
267     #     生成名字时是一个字一个字来的, 每次前向传播只有1个样本, BN已经没有“batch”可以统计了,
268     #     所以它必须退回去用训练过程中累积的running_mean和running_var。
269     #     注意: 只有BatchNorm1d才有training属性, Linear和Tanh没有, 所以要用isinstance筛一下
270     for laver in layers:

```



```

316
317 use()
318 sys.exit(0)
319
320
321
322 #=====开始为了调试而打印图像=====
323 # (1) 这一张看前向信号（激活值）是否健康，具体说是看tanh层的输出有没有过饱和。
324 #     1.这个直方图的横坐标是：tanh层的输出值，也就是激活值。
325 #         因为tanh的值域是(-1, 1)，所以横轴大致落在 -1 到 1 之间，形状是"两头被压扁的钟形"。
326 #     2.纵坐标是：概率密度分布。torch.histogram(..., density=True) 会把直方图归一化成面积=1，
327 #         这样不同层之间比较的是"形状"，而不是"样本数量"。
328 #     3.图中有很多条曲线，分别表示每一层tanh输出的激活值分布。
329 #         注意标签用的是layer在layers列表里的下标（每三件套 Linear,BN,Tanh 一组），所以是 2、5、8、11、14。
330 #     4.虽然每一tanh层输出是一个tensor（32个样本 × 100个神经元 = 3200个数），
331 #         但是统计对象是这个tensor里面各个分量的值，也就是把这3200个数展平后一起统计。
332 #     5.判断过饱和看print里的 saturated，它统计的是 |激活值| > 0.97 的分量所占的百分比。
333 #         为什么阈值取0.97？因为  $\tanh'(x) = 1 - \tanh(x)**2$ ，
334 #         当输出值是0.97时，导数只剩下  $1 - 0.97**2 \approx 0.06$ ，
335 #         也就是梯度传到这里只能按约6%的比例继续往回走；输出到0.99就只剩2%了。
336 #         所以这个百分比越高，说明越多的神经元被推到了tanh的平缓区，梯度会被压没。
337 #     6.健康的标志：
338 #         mean  $\approx 0$           — 上一层的BN把激活值正负对称地归一化了
339 #         std  $\approx 0.65$       — 标准正态输入经过tanh后，标准差大约就是0.63；关键是它要逐层基本不变
340 #         saturated 很低 — 只有几个百分点，说明绝大多数神经元工作在tanh的线性区
341 #         反过来，如果std逐层递减，就是"方差塌缩"，是梯度消失的前兆。
342 plt.figure(figsize=(20, 4)) # width and height of the plot
343 legends = []
344 for i, layer in enumerate(layers[:-1]): # note: exclude the output layer
345     if isinstance(layer, Tanh):
346         t = layer.out
347         print('layer %d (%10s): mean %+.2f, std %.2f, saturated: %.2f%%' % (i, layer.__class__.__name__, t.mean(), t.std(), (t.abs() > 0.97).float().mean()))
348         hy, hx = torch.histogram(t, density=True)
349         plt.plot(hx[:-1].detach(), hy.detach())
350         legends.append(f'layer {i} ({layer.__class__.__name__})')
351 plt.legend(legends);
352 plt.title('activation distribution')
353
354
355
356 # (2) 这一张看反向信号（梯度）是否健康。
357 #     1.数学上，tanh层的叠加会导致梯度的衰减。
358 #         tanh层的链式法则，会导致输入xin和输出xout有这样的关系：L表示Loss，xin和xout表示向量的一组对应分量
359 #          $dL/dx_{in} = (dL / d x_{out}) * (1 - x_{out}**2)$ ，
360 #         天然乘一个恒小于等于1的数。所以xin的grad一定小于等于xout

```

```

361 # 2.这张图的横坐标是：梯度值。
362 # 3.纵坐标是：概率密度分布，或者说是频率值
363 # 4.图中有很多个直线，分别表示每一层tanh输出的梯度分布
364 # 5.虽然输出是一个tensor，但是统计对象是这些tensor里面各个分量的梯度。
365 plt.figure(figsize=(20, 4)) # 定义图的大小
366 legends = []
367 for i, layer in enumerate(layers[:-1]): #最后一层不参与循环
368     if isinstance(layer, Tanh):#判断 layer 是不是 Tanh 类的实例。
369         t = layer.out.grad#如果是tanh的layer，那么就取出变量
370         print('layer %d (%10s): mean %+f, std %e' % (i, layer.__class__.__name__, t.mean(), t.std()))
371         hy, hx = torch.histogram(t, density=True)
372         plt.plot(hx[:-1].detach(), hy.detach())
373         legends.append(f'layer {i} ({layer.__class__.__name__})')
374 plt.legend(legends);
375 plt.title('gradient distribution')
376
377
378
379 # (3) 这一张看参数（权重矩阵）的梯度是否健康，也就是"反向传播传到最后，落到参数身上的梯度有多大"。
380 # 1.横坐标是：参数的梯度值，也就是 p.grad 里面各个分量的值。
381 # 2.纵坐标是：概率密度分布（同样是density=True归一化过）。
382 # 3.只挑 ndim == 2 的参数画图，也就是矩阵形状的权重：C 和 6 个 Linear 的 weight，一共7条曲线。
383 # BN的gamma/beta是一维的，不画：一是它们的每个分量对应"一个特征"，看分布意义不大；
384 # 二是它们初值全相同（gamma全1、beta全0），std=0，算前面的比值时会得到inf。
385 # 4.最关键的是print出来的 grad:data ratio = grad.std() / data.std(),
386 # 这是一个无量纲的比值，表示"梯度的大小相对于参数本身的大小"。
387 # 它决定了每一步更新的相对步长，经验上希望它落在 1e-3 附近。
388 # 5.注意这个梯度是"最后一次"前向-反向算出来的（循环里每步都会重新计算并覆盖），
389 # 所以它是训练结束那一刻的快照。
390 # 6.健康的标志：
391 # 各层梯度的量级不要相差太多（不要出现前面1e-2、后面1e-6这种数量级的落差）；
392 # grad:data ratio 在 1e-3 上下，同一数量级即可。
393 # 如果某个ratio特别大，说明该层学习率过高、会被一步打乱；特别小则说明该层学不动。
394 plt.figure(figsize=(20, 4)) # width and height of the plot
395 legends = []
396 for i,p in enumerate(parameters):
397     t = p.grad
398     if p.ndim == 2:
399         print('weight %10s | mean %+f | std %e | grad:data ratio %e' % (tuple(p.shape), t.mean(), t.std(), t.std() / p.std()))
400         hy, hx = torch.histogram(t, density=True)
401         plt.plot(hx[:-1].detach(), hy.detach())
402         legends.append(f'{i} {tuple(p.shape)}')
403 plt.legend(legends)
404 plt.title('weights gradient distribution');
405

```

```

406
407
408 # (4) 这一张看学习率是否合适一把每一次iteration算出来的"参数变化率"画成随时间的曲线。
409 # 1.横坐标是: iteration (训练步数, 这里因为break, 实际只有1001步)。
410 # 2.纵坐标是: 每一个参数矩阵的这个值:  $\log_{10}(\text{std}(\text{lr} * \text{p.grad}) / \text{std}(\text{p.data}))$ 。
411 # 我们按照当前神经网络的设计, 假设只有19个参数矩阵。然后其中只有一部分是二维的。假设有7个
412 # 那么我们这七个tensor, 每一次iteration都会计算一次 $(\text{lr} * \text{p.grad}).\text{std}() / \text{p.data}.\text{std}()).\log_{10}()$ 
413 # 3.所以整体图显示的就是, 有七条线, 每一条线都表示一个二维tensor的学习率, 关于i的变化趋势
414 # 3.注意  $\text{ud}[\text{j}][\text{i}]$  是转置: ud本身是  $\text{ud}[\text{step}][\text{param}]$ ,
415 # 而画图想让"某一个参数随时间的演变"成为一条曲线, 所以把参数这一维提到外层。
416 # 4.只画  $\text{ndim} == 2$  的参数, 所以图上只有7条曲线 (C 和 6 个 Linear 的 weight)。
417 # 5.那条黑色的直线是  $y = -3$  的基准线:  $\log_{10} \approx -3$  表示每步把权重改动约千分之一 ( $1e-3$ )。
418 # 这是 Karpathy 的经验法则一比值稳定贴着  $-3$ , 说明学习率合适。
419 # 远高于  $-3$  (比如  $-1$ ) 说明学习率太大, 参数会被一步打乱、训练震荡;
420 # 远低于  $-3$  说明学习率太小, 参数几乎不动、学得很慢。
421 # 6.为什么用 $\log_{10}$ : 不同参数、不同阶段的比值会跨越好几个数量级, 取对数后才好在同一张图上比较。
422 plt.figure(figsize=(20, 4))
423 legends = []
424 for i,p in enumerate(parameters):
425     if p.ndim == 2: #我们要研究的参数张量都是两个维度的。也就是C矩阵和线性层的W, B矩阵
426         plt.plot([ud[j][i] for j in range(len(ud))])
427         legends.append('param %d' % i)
428 plt.plot([0, len(ud)], [-3, -3], 'k') # these ratios should be  $\sim 1e-3$ , indicate on plot
429 plt.legend(legends);
430 # (5) 我们发现, 最后一个linear的W矩阵, 学习率的值比较高, why?
431
432
433
434 @torch.no_grad() # this decorator disables gradient tracking
435 def split_loss(split):
436     x,y = {
437         'train': (Xtr, Ytr),
438         'val': (Xdev, Ydev),
439         'test': (Xte, Yte),
440     }[split]
441     emb = C[x] # (N, block_size, n_embd)
442     x = emb.view(emb.shape[0], -1) # concat into (N, block_size * n_embd)
443     for layer in layers:
444         x = layer(x)
445     loss = F.cross_entropy(x, y)
446     print(split, loss.item())
447
448 # put layers into eval mode
449 for layer in layers:
450     layer.training = False

```



```

496 plt.figure(figsize=(10, 5))
497 # plot 0
498 plt.plot([-6,6], [0,0], 'k')
499 # plot the mean and std
500 xx = np.linspace(-6, 6, 100)
501 plt.plot(xx, stats.norm.pdf(xx, mu, sig), 'b')
502 xx = np.linspace(-6, 6, 100)
503 plt.plot(xx, stats.norm.pdf(xx, 0, 1), 'r')
504 # plot little lines connecting input and output
505 for i in range(len(x)):
506     plt.plot([x[i],y[i]], [1, 0], 'k', alpha=0.2)
507 # plot the input and output values
508 plt.scatter(x.data, torch.ones_like(x).data, c='b', s=100)
509 plt.scatter(y.data, torch.zeros_like(y).data, c='r', s=100)
510 plt.xlim(-6, 6)
511 # title
512 plt.title('input mu %.2f std %.2f' % (mu, sig))
513
514 interact(normshow, x0=(-30,30,0.5));
515
516
517
518 # Linear: activation statistics of forward and backward pass
519
520 g = torch.Generator().manual_seed(2147483647)
521
522 a = torch.randn((1000,1), requires_grad=True, generator=g) # a.grad = b.T @ c.grad
523 b = torch.randn((1000,1000), requires_grad=True, generator=g) # b.grad = c.grad @ a.T
524 c = b @ a
525 loss = torch.randn(1000, generator=g) @ c
526 a.retain_grad()
527 b.retain_grad()
528 c.retain_grad()
529 loss.backward()
530 print('a std:', a.std().item())
531 print('b std:', b.std().item())
532 print('c std:', c.std().item())
533 print('----')
534 print('c grad std:', c.grad.std().item())
535 print('a grad std:', a.grad.std().item())
536 print('b grad std:', b.grad.std().item())
537
538
539
540 # Linear + BatchNorm: activation statistics of forward and backward pass

```

